

Key-Value Stores

Ilyas Tachakor | Nick Albrecht

Sommersemester 2026

Agenda

01 Motivation

10 Min

02 Konzeptuelles Modell

10 Min

03 Datenstruktur

15 Min

04 Query-Modell (Redis-Python)

15 Min

05 Architektur

10 Min

06 Amazon Dynamo

30 Min

07 Ethik & Governance

10 Min

08 Praktische Demo

30 Min

09 Challenge

20 Min

10 Quiz

10 Min

11 Diskussion

10 Min

01

Motivation

*Warum Key-Value Stores? Datenmodell und Anwendungsfälle --
10 Minuten*

Warum brauchen wir Alternativen zu relationalen Datenbanken?

Skalierungsprobleme

Vertikale Skalierung (teurere Hardware) ist begrenzt. Horizontale Skalierung mit JOINS über viele Tabellen ist extrem komplex.

Unnötige Komplexität

Viele Services brauchen nur: Schlüssel rein, Wert raus. ACID-Transaktionen, Schema-Definitionen und SQL-Parsing verursachen Overhead.

Verfügbarkeits-Tradeoff

Relationale Replikation priorisiert Konsistenz. Bei Netzwerkausfällen werden Daten lieber unavailable als inkonsistent.

Das Key-Value Datenmodell

KEY

VALUE

user:1001

-->

`{"name": "Anna", "age": 29}`

session:abc

-->

`"tok_98afbd12c4e7"`

price:item42

-->

12.99

config:theme

-->

`"dark-mode"`

Anwendungsfälle für Key-Value Stores

High-Performance Caching

Seiteninhalte und API-Antworten im RAM speichern.
Beispiel: Redis als Cache-Layer vor einer SQL-Datenbank.

Session Management

User-Sessions speichern: key = session_id,
value = User-Objekt (Login-Status, Sprache, Warenkorb).

Warenkorb (Shopping Cart)

cart:user_id --> JSON mit Artikeln.
Genau so setzt Amazon es mit Dynamo ein.

Echtzeit-Anwendungen

Aktienkurse, Sport-Scores, IoT-Sensordaten.
key = AAPL, value = 182.50 -- jede Sekunde aktualisiert.

Feature Flags und A/B-Tests

flag:dark_mode --> true/false.
Schnell änderbar ohne Deployment.

Geschichte und Entstehung

1990er



Relationale Datenbanken dominieren. ACID überall.

2003



Google: Bigtable -- verteilter Speicher für strukturierte Daten.

2007



Amazon Dynamo Paper (SOSP) -- hochverfügbarer KV-Store für E-Commerce.

2009



Redis 1.0 erscheint -- In-Memory KV-Store mit reichen Datentypen.

2010+



Memcached, Riak, Voldemort, DynamoDB verbreiten sich.

heute



KV-Stores als Kernkomponente jeder modernen Cloud-Architektur.

02

Konzeptuelles Modell

Kernstruktur, Analogien und Abgrenzung -- 10 Minuten

Analogie: Das Wörterbuch

Kernprinzipien

Eindeutiger Key

Jeder Key existiert genau einmal

O(1) Zugriff

Hash-Funktion garantiert konstante Zugriffszeit

Opaque Value

DB kennt Struktur des Values nicht (schema-less)

Keine Relationen

Kein JOIN, kein Schema, kein Foreign Key

Wörterbuch-Prinzip

Schlüssel ist einzigartig. Suche erfolgt direkt, ohne sequentielles Durchsuchen.

Apfel

Frucht, rund, rot oder grün

Datenbank

sehr schnell, gut skalierbar

Redis

Remote Dictionary Server

Cache im Key-Value-Store

Funktion

Cache speichert im RAM (schnell aber begrenzt)
Lösung: Neue Cache Schicht

Was wird gecacht?

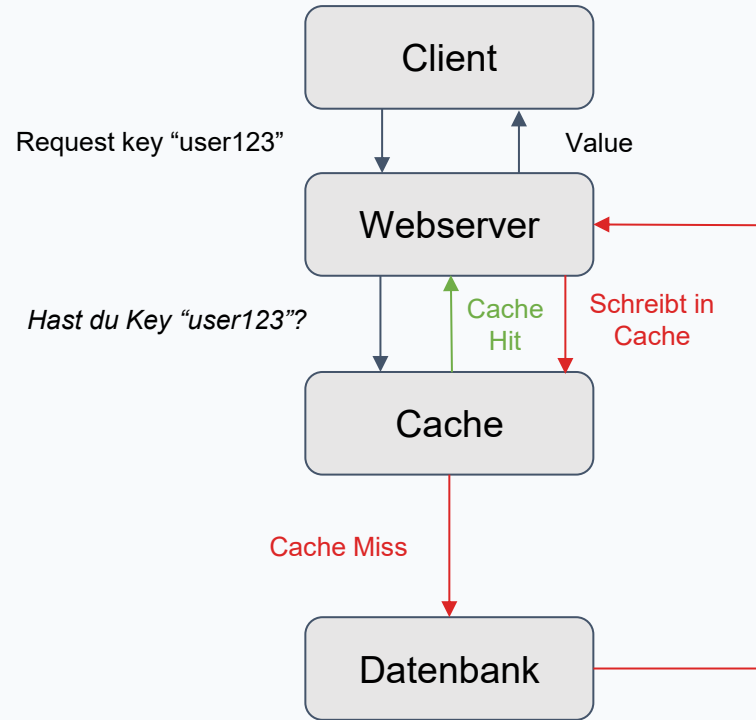
Hot Data, komplexe Querries, zuletzt Verwendete Daten

Wann wird Cache gelöscht?

TTL, Eviction (LRU, LFU), Invalidation, restart

Cache-Aside Pattern

Webserver entscheidet woher die Daten kommen



Key-Value Store vs. Document Store

Key-Value Store

Value ist für die DB ein opaker Blob

Lookup nur per exaktem Key

Extrem schnell und einfach

Ideal: Caching, Sessions, Counters

Redis, Memcached, DynamoDB

Document Store

DB versteht die Dokument-Struktur

Queries auf Felder im Dokument möglich

Flexibler, aber höherer Overhead

Ideal: Produktkataloge, User-Profile

MongoDB, CouchDB, Firestore

03

Datenstruktur

Physische und logische Speicherung, Schema-Flexibilität -- 15 Minuten

Logische Struktur: Die Hash Table

Key	Hash(Key)	Bucket	Value
user:1001	0x3F4A	#2	{"name": "Anna", "email": "a@ex.com"}
session:abc	0x12BE	#7	"tok_98afbd12"
price:item42	0xA1C3	#4	29.99
flag:beta	0x77F2	#1	true

Physische Speicherung: In-Memory vs. On-Disk

In-Memory

Daten liegen vollständig im RAM

Mikrosekunden-Latenz (Sub-ms)

Persistenz optional via RDB-Snapshots

Ideal für Caching und Sessions

Begrenzter Speicher: RAM ist teuer

On-Disk

Daten auf SSD/HDD (LSM-Tree oder B-Tree)

Millisekunden-Latenz

Persistenz ist garantiert

Ideal für grosse Datenmengen (TB-Bereich)

Langsamer als In-Memory, aber günstiger

Schema-Flexibilität: Schema-less

Python

```
# Dieselbe Datenbank -- völlig unterschiedliche Value-Strukturen
import redis
r = redis.Redis()

r.set("user:1001", '{"name":"Anna","age":29,"premium":true}')
r.set("user:1002", '{"name":"Bob","city":"Berlin"}')
r.set("user:1003", '{"name":"Cara","tags":["admin","beta"]}')
```

Kein Schema nötig -- jeder Value darf anders strukturiert sein

Risiko

Schema-less bedeutet: Die Applikation muss Datenqualität selbst sicherstellen. Tippfehler in Keys führen zu stillen Datenverlusten.

Unterschied zu Document Stores

MongoDB kann Schema-Validierung erzwingen und auf Felder innerhalb des Dokuments querien. KV-Stores behandeln den Value als Black Box.

04

Query-Modell

Redis-Python-Befehle und Grundoperationen -- 15 Minuten

Grundoperationen: SET, GET, DELETE

SET (Schreiben)

```
import redis
r = redis.Redis(host="localhost", port=6379, decode_responses=True)

r.set("session:user42", '{"lang":"de","theme":"dark"}')
r.set("counter:visits", 0)
r.set("price:item99", 19.99, ex=3600) # ex = TTL in Sekunden
```

Python

GET (Lesen)

```
val = r.get("session:user42")
print(val) # --> '{"lang":"de","theme":"dark"}'

missing = r.get("nonexistent")
print(missing) # --> None
```

Python

DELETE (Löschen)

```
deleted = r.delete("session:user42")
print(deleted) # --> 1 (Anzahl gelöschter Keys)

r.delete("key1", "key2", "key3") # Mehrere Keys auf einmal
```

Python

Redis-Datentypen in Python

String / Counter

```
r.set("hits", 42)
r.incr("hits")      # --> 43
r.get("hits")       # --> "43"
```

List (Queue/Stack)

```
r.lpush("queue", "job1", "job2")      # --> queue = ["job2", "job1"]
r.rpop("queue")                       # --> "job1"
r.lrange("queue", 0, -1)               # --> ["job2"]
```

Hash (verschachtelte KV)

```
r.hset("user:1", mapping={"name": "Anna", "city": "Berlin"})
r.hget("user:1", "name") # --> "Anna"
r.hgetall("user:1")      # --> {"name": "Anna", "city": "Berlin"}
```

Set (Unique Values)

```
r.sadd("tags", "redis", "python", "redis")
r.smembers("tags") # --> {"redis", "python"}
r.sinter("tags", "other_tags")
```

Sorted Set (Rankings)

```
r.zadd("scores", {"Anna": 100, "Bob": 85})
r.zrange("scores", 0, -1, withscores=True) #Bob,85 ; Anna,100
r.zrange("scores", 0, -1, desc=True, withscores=True) #Anna,100 ; Bob,85
r.zrank("scores", "Anna") # --> 1
r.zrevrank("scores", "Anna") #--> 0
```

Redis-Python vs. SQL: Denken in Keys statt Tabellen

SQL

```
SELECT name, email
FROM users
WHERE id = 1001;

INSERT INTO sessions
  (id, token, user_id)
VALUES
  ('abc', 'tok123', 1001);

DELETE FROM sessions
WHERE id = 'abc';
```

Redis-Python

```
user = r.hgetall("user:1001")
# --> {"name": "Anna", "email": "a@e.com"}

r.hset("session:abc",
      mapping={"token": "tok123",
              "user_id": "1001"})

r.delete("session:abc")
```

Wichtige weitere Redis-Python-Befehle

TTL und Expiry

```
r.set("cache:page", "<html>...</html>", ex=3600) # 1 Stunde TTL
print(r.ttl("cache:page")) # --> 3598
r.persist("cache:page") # TTL entfernen
r.expire("cache:page", 7200) # Neuen TTL setzen
```

Python

Multi-Operationen

```
r.mset({"k1": "v1", "k2": "v2", "k3": "v3"}) # Atomar
values = r.mget("k1", "k2", "k3")
print(values) # --> ["v1", "v2", "v3"]
```

Python

Existenz und Pattern-Suche

```
r.exists("session:abc") # --> 1 (True) oder 0 (False)
keys = r.keys("user:*") # Alle User-Keys (Vorsicht: blockiert, langsam bei großen Datenmengen)
for key in r.scan_iter("user:*"): # Produktionsfreundlich
    print(key)
```

Python

05

Architektur

Deployment, Replikation, Partitionierung und Tools -- 10 Minuten

Single Node vs. Verteiltes System

Single Node

- + Einfache Einrichtung und Verwaltung
- + Keine Konsistenz-Probleme
- Single Point of Failure
- Begrenzte Kapazität (RAM/Disk)

Geeignet für Entwicklung, kleine Projekte

Verteiltes System (Cluster)

- + Horizontale Skalierung (Nodes hinzufügen)
- + Hochverfügbarkeit durch Replikation
- + Partitionierung verteilt Daten und Last
- Konsistenz-Tradeoffs (CAP-Theorem)

Nötig für Produktion mit hohen Anforderungen

Replikation und Partitionierung

Replikation

Dieselben Daten auf mehreren Nodes:

- Primary --> Replica(s) Replikation
- Lese-Skalierung via Replicas
- Automatischer Failover: Replica wird Primary
- Beispieltool: Redis Sentinel

Partitionierung (Sharding)

Verschiedene Daten auf verschiedenen Nodes:

- Key-basiertes Sharding
- Range Partitioning
- Virtual Nodes für gleichm. Verteilung
- Beispieltool: Dynamo

CAP-Theorem

Ein verteiltes System kann maximal 2 von 3 garantieren: Consistency, Availability, Partition Tolerance.

Vergleich zu Document Stores: MongoDB nutzt ebenfalls Sharding und Replikation, routet Queries jedoch basierend auf der Dokument-Struktur, nicht nur nach Key-Hash.

Open-Source Tools und Cloud-Angebote

Redis

In-Memory, vielfältige Datentypen, De-facto-Standard.

Memcached

Simpler reiner In-Memory-Cache. Kein Clustering built-in.

RocksDB

Embedded KV (LSM-Tree) für persistente On-Disk-Daten.

AWS

ElastiCache (Redis/Memcached), DynamoDB, DAX

Azure

Azure Cache for Redis, Cosmos DB (KV-API), Table Storage

GCP

Cloud Memorystore, Firestore, Bigtable

Links die drei wichtigsten Open-Source-Tools: Redis (der Standard), Memcached (simpler, reiner Cache), RocksDB (On-Disk fuer grosse Datenmengen).

Rechts die Cloud-Angebote der drei grossen Provider. Wichtig: In der Praxis nutzt man oft Managed Services statt selbst betriebener Instanzen.

Hinweis auf die Demo: Wir nutzen redis-py mit einem lokalen Redis-Server. Die Teilnehmer sollen das Package installieren: `pip install redis`.

06

Amazon Dynamo

Real-World-Fallstudie (SOSP 2007) -- 30 Minuten

Amazon Dynamo: Kontext

Amazon.com im Jahr 2007

Weltweit agierendes Unternehmen. Müssen viele Kundenanfragen gleichzeitig verarbeiten.

Kernanforderung:

Kunden müssen ihren Warenkorb jederzeit nutzen können -- auch wenn Disks ausfallen, Netzwerke flackern oder ganze Datenzentren zerstört werden.

Hunderte Services

Dezentral und lose gekoppelt

Einfache Queries

Nur Primary-Key-Zugriff nötig

Commodity Hardware

Keine teuren Enterprise-Server

Always-On

24/7 Verfügbarkeit ohne Downtime

Dynamo: Design-Prinzipien

Always Writeable

Schreiboperationen werden niemals abgelehnt -- auch bei Ausfällen. Verfügbarkeit hat Vorrang vor Konsistenz.

Eventual Consistency

Alle Replicas werden irgendwann konsistent. Konflikte werden später aufgelöst -- nicht beim Schreiben.

Dezentralisierung

Keine Master-Knoten. Jeder Node hat dieselben Verantwortlichkeiten (Symmetrie). Kein Single Point of Failure.

Inkrementelle Skalierbarkeit

Nodes können einzeln hinzugefügt oder entfernt werden, ohne den gesamten Cluster zu beeinflussen.

Heterogenität

Die Lastverteilung passt sich der Kapazität einzelner Server an -- stärkere Nodes übernehmen mehr.

Dynamo: System Interface

Nur zwei Operationen -- bewusst minimalistisch:

get(key)

Gibt ein Objekt (oder mehrere konfliktäre Versionen) plus einen Context zurück.

put(key, context, object)

Schreibt das Objekt auf die zuständigen Nodes. Context enthält Versions Informationen.

Opake Byte-Arrays

Key und Value werden nicht interpretiert

MD5-Hashing

128-Bit-Identifizier zur Node-Zuordnung

Kleine Objekte

Typisch < 1 MB (Warenkorb, Sessions)

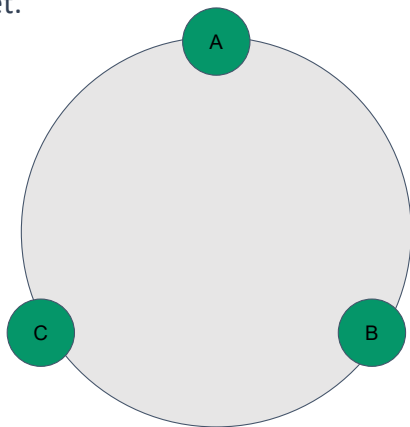
Kein ACID

Konsistenz bewusst aufgeweicht fuer Availability

Dynamo: Consistent Hashing und Virtual Nodes

Consistent Hashing

Der Hashraum wird als Ring (Kreis) modelliert. Jeder Node erhält eine Position auf dem Ring. Ein Key wird gehasht und dem nächsten Node im Uhrzeigersinn zugeordnet.



Virtual Nodes

Problem: Zufällige Positionen führen zu ungleichmäßiger Lastverteilung.

Lösung: Jeder physische Node erhält mehrere Positionen (Tokens) auf dem Ring. Ein physischer Node erscheint als mehrere virtuelle Nodes.

Vorteile:

- Gleichmäßige Lastverteilung
- Bei Ausfall wird Last verteilt
- Stärkere Nodes: mehr Virtual Nodes
- Heterogenität wird berücksichtigt

Dynamo: Klassisches Hashing

Beispiel

12 Pairs, 3 Server (A,B,C)

Klassisches Hashing

Node: $\text{hash}(\text{key}) \bmod 3$

Neuer Node

Node: $\text{hash}(\text{key}) \bmod 4$

hash(key)	Server
1	A
2	B
3	C
4	A
5	B
6	C
7	A
8	B
9	C
10	A
11	B
12	C



hash(key)	Server
1	A
2	B
3	C
4	D
5	A
6	B
7	C
8	D
9	A
10	B
11	C
12	D

Dynamo: Consistent Hashing

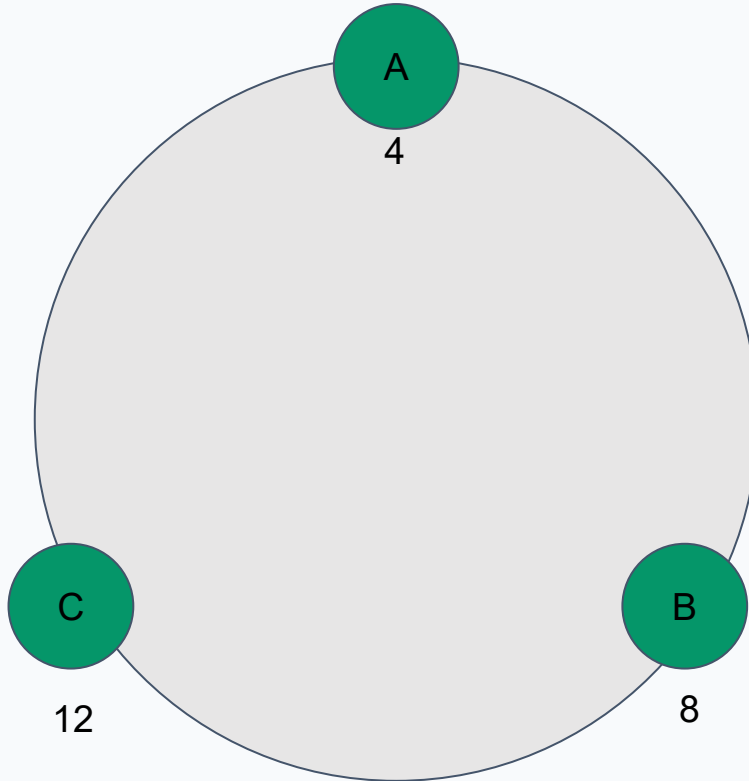
Beispiel

12 Pairs

Consistent hashing

Node: Bereich auf dem
Ring

Node	Positionen
A (4)	1-4
B (8)	5-8
C (12)	9-12



hash(key)	Server
1	A
2	A
3	A
4	A
5	B
6	B
7	B
8	B
9	C
10	C
11	C
12	C

Dynamo: Consistent Hashing

Beispiel

12 Pairs

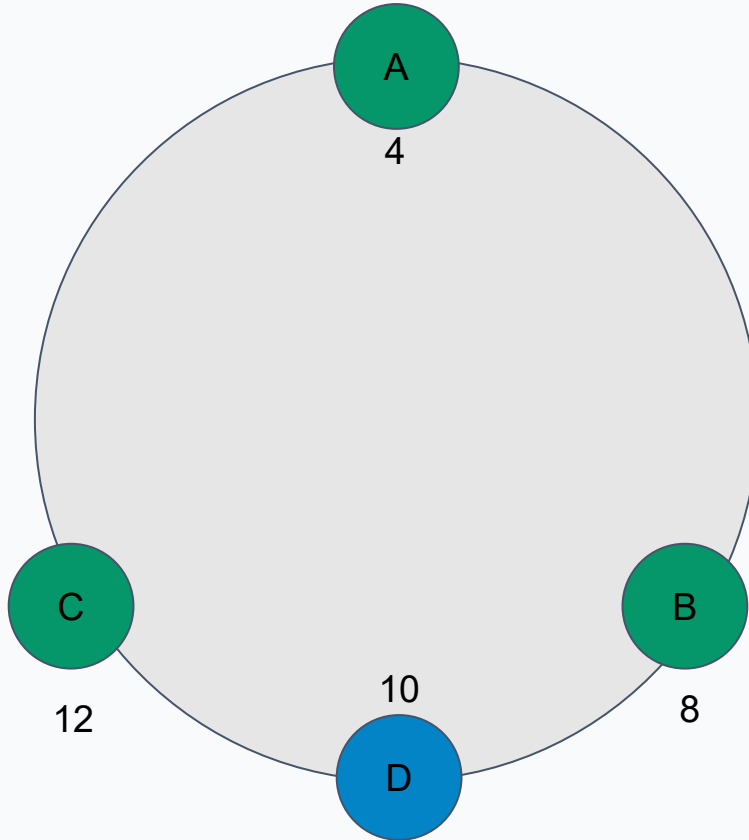
Consistent hashing

Node: Bereich auf dem
Ring

Neuer Node

Stelle 10

Node	Positionen
A (4)	1-4
B (8)	5-8
C (12)	11-12
D (10)	9-10



hash(key)	Server
1	A
2	A
3	A
4	A
5	B
6	B
7	B
8	B
9	D
10	D
11	C
12	C

Dynamo: Consistent Hashing

Beispiel

12 Pairs

Consistent hashing

Node: Bereich auf dem
Ring

Neuer Node

Stelle 10

hash(key)	Server
1	A
2	A
3	A
4	A
5	B
6	B
7	B
8	B
9	C
10	C
11	C
12	C



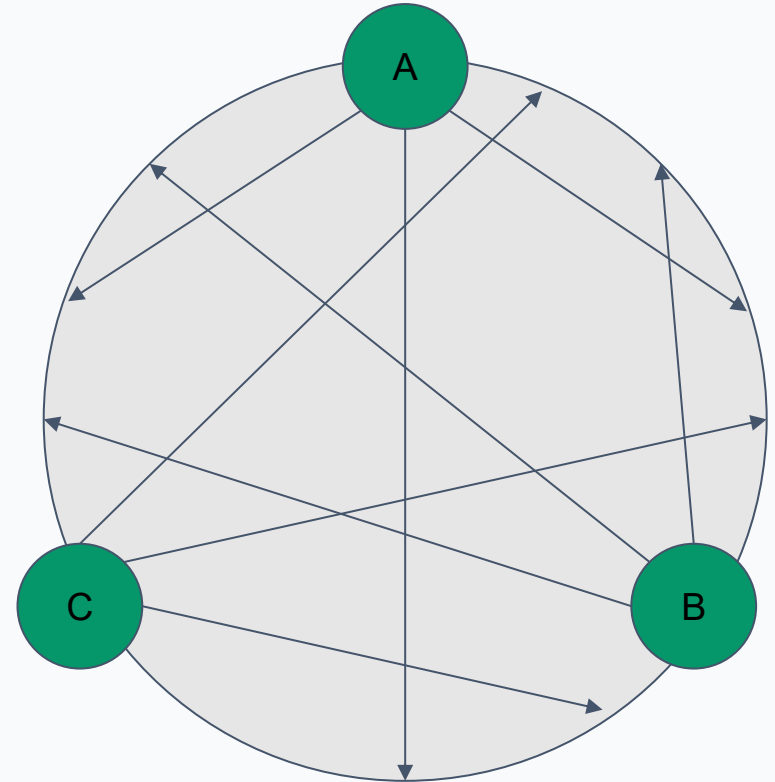
hash(key)	Server
1	A
2	A
3	A
4	A
5	B
6	B
7	B
8	B
9	D
10	D
11	C
12	C

Dynamo: Virtual Nodes

Virtual Nodes

Zufällig verteilte virtuelle
Nodes eines Servers

Data	Virtual Node	Physischer Node
1	B1	B
2	B1	B
3	C1	C
4	A2	A
5	C3	C
6	A2	A
7	B3	B
...	...	...



Dynamo: Replikation mit Preference Lists

N Replicas

Jeder Key wird auf N Hosts repliziert (typisch $N=3$).
Coordinator speichert lokal + repliziert auf $N-1$ Nachbarn im Ring.

Preference List

Geordnete Liste der Nodes für einen Key.
Enthält $> N$ Einträge, um Ausfälle abzufangen.

Beispiel ($N=3$)

Key K --> Node B koordiniert, speichert lokal.
Repliziert auf C und D. Preference List: [B, C, D, E, ...].

Multi-Datacenter

Preference List verteilt Nodes über mehrere Datenzentren.
Ganzer DC-Ausfall wird ohne Datenverlust überlebt.

Dynamo: Versionierung mit Vector Clocks

Eventual Consistency

put() kehrt zurück, bevor alle Replicas aktualisiert sind. Mehrere Versionen können gleichzeitig existieren.

Syntaktische Rekonziliation (Vector Clocks)
System erkennt automatisch die neueste Version.

Semantische Rekonziliation
Bei Konflikten: Applikation merged Versionen.

*Beispiel: Shopping Cart merged
verschiedene Warenkorb-Versionen.*

Vector Clocks

```
A: [2|1|0]
B: [1|1|0]    <-Anfrage (B=Coordinator node)
C: [1|0|0]
```

A dominiert -> Wert genommen

```
A: [2|0|1]
B: [1|1|0]    <-Anfrage (B=Coordinator node)
C: [1|0|0]
```

keine Dominanz -> Konflikt, beide Versionen existieren parallel, Rekonziliation nötig

Node dominiert, wenn alle Werte von z.B. Node A $\geq$ B sind

Dynamo: Quorum (N, R, W) und Sloppy Quorum

Quorum-Parameter

N = Anzahl Replicas (typisch: 3)

R = Schreib Quorum

W = Lese Quorum

$R + W > N$ --> Quorum. Standard: (3, 2, 2)

Sloppy Quorum + Hinted Handoff

Kein striktes Quorum! Reads/Writes gehen an die ersten N gesunden Nodes.

Node A fällt aus --> Replica geht an Node D (mit Hint). Nach Recovery: Nachlieferung.

Konfigurierbare Tradeoffs:

W = 1

Max. Verfügbarkeit, Risiko für Inkonsistenz

W = N

Max. Konsistenz, niedrige Verfügbarkeit

R = 1, W = N

High-Performance Read Engine (z.B. Produktkatalog)

Dynamo: Anti-Entropy und Membership

Merkle Trees (Anti-Entropy)

Zur effizienten Replikat-Synchronisation. Ein Hash-Baum, dessen Blätter die Hashes einzelner Keys sind. Elternknoten hashen ihre Kinder.

Vergleich: Zwei Nodes tauschen Root-Hashes aus. Gleich? Alles synchron. Unterschiedlich? Nur die betroffenen Branches werden tiefer verglichen.

Minimiert Datentransfer und Disk-Reads.

Gossip-Protokoll

Membership-Informationen werden dezentral über ein Gossip-Protokoll verbreitet. Jede Sekunde kontaktiert ein Node einen zufällig gewählten Peer.

Seed-Nodes verhindern logische Partitionen im Ring. Failure Detection: Ein Node gilt als ausgefallen, wenn er auf Nachrichten nicht antwortet.

Vorteil: Kein zentrales Registry. Symmetrisch.

Read Repair

Nach einem Read wartet der Coordinator kurz auf späte Antworten. Enthalten diese veraltete Versionen, werden die betroffenen Nodes opportunistisch mit der neuesten Version aktualisiert. Das entlastet den Anti-Entropy-Prozess erheblich.

Dynamo: Nutzungsmuster

Business-Logic Reconciliation

Applikation führt eigene Merge-Logik aus. Beispiel: Shopping Cart merged verschiedene Warenkorb-Versionen.

Timestamp-based Reconciliation

Last-Write-Wins anhand physischer Timestamps. Einfacher, aber weniger präzise. Beispiel: Session-Management.

High-Performance Read Engine

$R=1$, $W=N$. Viele Reads, wenige Writes. Genutzt als autoritativer Cache für Produktkataloge.

Dynamo: Ergebnisse in Produktion

99.9995%

Verfügbarkeit

< 300ms

SLA (p99.9)

3 Replicas

(N, R, W) = (3, 2, 2)

0

Datenverluste

Holiday Season: Shopping Cart Service bediente Millionen Requests mit über 3 Mio. Checkouts pro Tag -- ohne Downtime.

Divergente Versionen sind selten: 99.94% der Anfragen sahen genau eine Version. Nur 0.00057% sahen zwei.

Client-driven Koordination senkte die p99.9-Latenz um mindestens 30ms gegenüber dem Load-Balancer-Ansatz.

Write-Buffering reduzierte die p99.9-Latenz um Faktor 5 bei nur 1.000 gepufferten Objekten.

Partitionierungsstrategie 3 (Q/S Tokens, feste Partitionen) erzielte die beste Lastverteilung und vereinfachte Archivierung.

07

Ethik & Governance

*Verantwortungsvoller Umgang mit Key-Value Stores -- 10
Minuten*

Datenschutz und DSGVO-Konformität

Frage: Welche personenbezogenen Daten können in einem Redis-Cache landen -- und wüsste man es?

Datenschutz und DSGVO-Konformität

Frage: Welche personenbezogenen Daten können in einem Redis-Cache landen -- und wüsste man es?

Datenpersistenz unklar

In-Memory KV-Stores ohne Persistenz: Bei Neustart sind Daten weg -- aber kein Audit-Trail, wann was gespeichert war.

Personenbezogene Daten

Session-IDs, User-Tokens, IP-Adressen, Suchverläufe -- oft ohne explizite Dokumentation gecached.

Datenlokalisierung

Cloud-Dienste (AWS ElastiCache) können Daten in andere Länder replizieren. DSGVO verlangt Datenhaltung im EWR.

Recht auf Löschung (Art. 17 DSGVO)

TTL-basiertes Ablaufen von Keys ist kein vollständiger Ersatz für aktive Löschung personenbezogener Daten.

Bias durch Caching-Mechanismen

Das Feedback-Loop-Problem

Populärer Content wird gecached --> schnellere Auslieferung --> wird häufiger angeklickt --> wird noch stärker gecached. Seltene Inhalte haben einen systematischen Nachteil.

Beispiel: Nachrichten-Plattform

Top-100-Artikel gecached, Nischen-Themen nicht. Minoritäre Inhalte verschwinden.

Beispiel: E-Commerce

Empfehlungen basieren auf gecachten Bestsellern. Neue oder spezialisierte Produkte benachteiligt.

Gegenmassnahmen:

Cache-Strategie bewusst designen (nicht nur LRU)

Monitoring: Welche Inhalte werden nie gecached?

Zufällige Exploration einbauen (Epsilon-Greedy)

Regelmäßige Audits der Cache-Hit-Verteilung

Datenqualität bei Schema-less Systemen

Risiken

Falsche Datentypen

String statt Integer im Value

Tippfehler in Keys

usre:1001 statt user:1001 --> stiller Verlust

Inkonsistente Strukturen

Manche Objekte mit email-Feld, andere ohne

Keine referenzielle Integrität

Gelöschte User bleiben als Waisen

Gegenmassnahmen

Key-Namenskonventionen

z.B. user:{id}:profile dokumentieren

Schema-Validierung

Pydantic, JSON Schema im Application-Layer

Monitoring

Anomalie-Erkennung auf Key-Patterns

Data-Quality-Audits

Regelmässig prüfen und bereinigen

In Document Stores (MongoDB) kann Schema-Validierung direkt in der DB erzwungen werden -- in KV-Stores nicht.

Erklärbarkeit und Auditierbarkeit

Wer hat wann welchen Wert gesetzt -- und warum?

Kein eingebautes Audit-Log

KV-Stores bieten typischerweise keine Query-History oder Daten-Lineage. Wenn ein Wert falsch ist, gibt es keinen DB-seitigen Hinweis, wer ihn wann geändert hat.

Keine Transaktions-Historie

Anders als relationale Datenbanken mit Transaction Logs oder Redo-Logs fehlt in vielen KV-Stores ein vollständiges Write-Ahead-Log für Compliance-Zwecke.

Regulatorische Anforderungen

DSGVO Art. 30 verlangt ein Verzeichnis von Vorbereitungstätigkeiten. Branchenregulierung (z.B. Finanz) fordert Audit-Trails für Datenänderungen.

Empfehlung

Applikationsseitiges Logging implementieren. Änderungen in einem separaten Audit-Stream festhalten (z.B. Redis Streams, Kafka, Elasticsearch).

Governance-Empfehlungen: Zusammenfassung

Key-Namenskonventionen

Dokumentierte Naming-Patterns (z.B. entity:id:field). Automatisierte Checks.

TTL-Policies

Personenbezogene Daten mit TTL versehen. Kein unendliches Caching von PII.

Verschlüsselung

At-rest und in-transit.

Audit-Logging

Application-Layer Logs für alle Schreiboperationen.

Monitoring und Alerting

Cache-Hit-Raten, Key-Verteilung, Memory-Nutzung. Alerts bei ungewöhnlichen Mustern.

Dokumentation

Data Dictionary pflegen: Welche Key-Patterns existieren, wer ist Owner, welche Daten stecken drin?

08

Praktische Demo

30 Minuten

09

Challenge

20 Minuten

10

Quiz

Wissensüberprüfung -- 10 Minuten

Frage 1

Was ist die Zeitkomplexität eines Key-Lookups in einem Key-Value Store?

Frage 1

Was ist die Zeitkomplexität eines Key-Lookups in einem Key-Value Store?

A

Quadratisch

Frage 1

Was ist die Zeitkomplexität eines Key-Lookups in einem Key-Value Store?

A

Quadratisch

B

Logarithmisch

Frage 1

Was ist die Zeitkomplexität eines Key-Lookups in einem Key-Value Store?

A

Quadratisch

B

Logarithmisch

C

Linear

Frage 1

Was ist die Zeitkomplexität eines Key-Lookups in einem Key-Value Store?

A

Quadratisch

B

Logarithmisch

C

Linear

D

Konstant

Frage 2

Welcher Python-Redis-Befehl setzt einen Key mit automatischer Ablaufzeit von 60 Sekunden?

Frage 2

Welcher Python-Redis-Befehl setzt einen Key mit automatischer Ablaufzeit von 60 Sekunden?

A

```
r.set("k", "v", timeout=60)
```

Frage 2

Welcher Python-Redis-Befehl setzt einen Key mit automatischer Ablaufzeit von 60 Sekunden?

A

`r.set("k", "v", timeout=60)`

B

`r.set("k", "v", ex=60)`

Frage 2

Welcher Python-Redis-Befehl setzt einen Key mit automatischer Ablaufzeit von 60 Sekunden?

A

`r.set("k", "v", timeout=60)`

B

`r.set("k", "v", ex=60)`

C

`r.insert("k", "v", ttl=60)`

Frage 2

Welcher Python-Redis-Befehl setzt einen Key mit automatischer Ablaufzeit von 60 Sekunden?

A

`r.set("k", "v", timeout=60)`

B

`r.set("k", "v", ex=60)`

C

`r.insert("k", "v", ttl=60)`

D

`r.put("k", "v", expire=60)`

Frage 3

Was unterscheidet einen Key-Value Store grundlegend von einem Document Store?

Was unterscheidet einen Key-Value Store grundlegend von einem Document Store?

A

KV-Stores sind immer schneller

Was unterscheidet einen Key-Value Store grundlegend von einem Document Store?

A

KV-Stores sind immer schneller

B

KV-Stores können kein JSON speichern

Was unterscheidet einen Key-Value Store grundlegend von einem Document Store?

A

KV-Stores sind immer schneller

B

KV-Stores können kein JSON speichern

C

Document Stores verstehen die Struktur des Values

Was unterscheidet einen Key-Value Store grundlegend von einem Document Store?

A

KV-Stores sind immer schneller

B

KV-Stores können kein JSON speichern

C

Document Stores verstehen die Struktur des Values

D

KV-Stores unterstützen kein Clustering

Amazon Dynamo priorisiert im CAP-Theorem welche Kombination?

Amazon Dynamo priorisiert im CAP-Theorem welche Kombination?

A

Consistency + Availability

Amazon Dynamo priorisiert im CAP-Theorem welche Kombination?

A

Consistency + Availability

B

Consistency + Partition Tolerance

Frage 4

Amazon Dynamo priorisiert im CAP-Theorem welche Kombination?

A

Consistency + Availability

B

Consistency + Partition Tolerance

C

Availability + Partition Tolerance

Frage 4

Amazon Dynamo priorisiert im CAP-Theorem welche Kombination?

A

Consistency + Availability

B

Consistency + Partition Tolerance

C

Availability + Partition Tolerance

D

Alle drei gleichzeitig

Amazon Dynamo priorisiert im CAP-Theorem welche Kombination?

- A** Consistency + Availability
- B** Consistency + Partition Tolerance
- C** Availability + Partition Tolerance
- D** Alle drei gleichzeitig

Erklärung: Dynamo ist ein AP-System: Writes werden nie abgelehnt. Konsistenz wird durch Eventual Consistency und applikationsseitige Rekonziliation erreicht.

Welches Verfahren nutzt Dynamo zur Versionierung und Konflikterkennung?

Welches Verfahren nutzt Dynamo zur Versionierung und Konflikterkennung?

A

Timestamps (Last-Write-Wins)

Welches Verfahren nutzt Dynamo zur Versionierung und Konflikterkennung?

A

Timestamps (Last-Write-Wins)

B

Two-Phase Commit

Welches Verfahren nutzt Dynamo zur Versionierung und Konflikterkennung?

A Timestamps (Last-Write-Wins)

B Two-Phase Commit

C Optimistic Locking

Welches Verfahren nutzt Dynamo zur Versionierung und Konflikterkennung?

A

Timestamps (Last-Write-Wins)

B

Two-Phase Commit

C

Optimistic Locking

D

Vector Clocks

Welches Verfahren nutzt Dynamo zur Versionierung und Konflikterkennung?

A

Timestamps (Last-Write-Wins)

B

Two-Phase Commit

C

Optimistic Locking

D

Vector Clocks

Erklärung: Vector Clocks sind Listen von (Node, Counter)-Paaren, die Kausalität zwischen Versionen erfassen. Parallele Branches bedeuten Konflikte.

Was ist der Standardwert für die Konfiguration (N, R, W) in Dynamo?

Was ist der Standardwert für die Konfiguration (N, R, W) in Dynamo?

A

(1, 1, 1)

Frage 6

Was ist der Standardwert für die Konfiguration (N, R, W) in Dynamo?

A

(1, 1, 1)

B

(3, 2, 2)

Frage 6

Was ist der Standardwert für die Konfiguration (N, R, W) in Dynamo?

A (1, 1, 1)

B (3, 2, 2)

C (5, 3, 3)

Frage 6

Was ist der Standardwert für die Konfiguration (N, R, W) in Dynamo?

A (1, 1, 1)

B (3, 2, 2)

C (5, 3, 3)

D (3, 1, 3)

Frage 6

Was ist der Standardwert für die Konfiguration (N, R, W) in Dynamo?

A (1, 1, 1)

B (3, 2, 2)

C (5, 3, 3)

D (3, 1, 3)

Erklärung: N=3 Replicas, R=2 für Reads, W=2 für Writes. Diese Konfiguration balanciert Performance, Konsistenz und Verfügbarkeit.

Welches DSGVO-Recht ist für In-Memory-Caching besonders herausfordernd?

Welches DSGVO-Recht ist für In-Memory-Caching besonders herausfordernd?

A

Recht auf Datenportabilität (Art. 20)

Welches DSGVO-Recht ist für In-Memory-Caching besonders herausfordernd?

A

Recht auf Datenportabilität (Art. 20)

B

Recht auf Auskunft (Art. 15)

Welches DSGVO-Recht ist für In-Memory-Caching besonders herausfordernd?

A

Recht auf Datenportabilität (Art. 20)

B

Recht auf Auskunft (Art. 15)

C

Recht auf Löschung (Art. 17)

Welches DSGVO-Recht ist für In-Memory-Caching besonders herausfordernd?

A

Recht auf Datenportabilität (Art. 20)

B

Recht auf Auskunft (Art. 15)

C

Recht auf Löschung (Art. 17)

D

Recht auf Widerspruch (Art. 21)

Frage 7

Welches DSGVO-Recht ist für In-Memory-Caching besonders herausfordernd?

A

Recht auf Datenportabilität (Art. 20)

B

Recht auf Auskunft (Art. 15)

C

Recht auf Löschung (Art. 17)

D

Recht auf Widerspruch (Art. 21)

Erklärung: TTL-basiertes Ablaufen ist kein Ersatz für aktive Löschung. Art. 17 (Right to be Forgotten) muss explizit implementiert werden.

Wie heisst das Verfahren, mit dem Dynamo bei temporären Ausfällen Replicas an Ersatz-Nodes weiterleitet?

Wie heisst das Verfahren, mit dem Dynamo bei temporären Ausfällen Replicas an Ersatz-Nodes weiterleitet?

A

Read Repair

Frage 8

Wie heisst das Verfahren, mit dem Dynamo bei temporären Ausfällen Replicas an Ersatz-Nodes weiterleitet?

A

Read Repair

B

Merkle Tree Sync

Frage 8

Wie heisst das Verfahren, mit dem Dynamo bei temporären Ausfällen Replicas an Ersatz-Nodes weiterleitet?

A

Read Repair

B

Merkle Tree Sync

C

Hinted Handoff

Frage 8

Wie heisst das Verfahren, mit dem Dynamo bei temporären Ausfällen Replicas an Ersatz-Nodes weiterleitet?

A

Read Repair

B

Merkle Tree Sync

C

Hinted Handoff

D

Gossip Forwarding

Frage 8

Wie heisst das Verfahren, mit dem Dynamo bei temporären Ausfällen Replicas an Ersatz-Nodes weiterleitet?

A

Read Repair

B

Merkle Tree Sync

C

Hinted Handoff

D

Gossip Forwarding

Erklärung: Bei Hinted Handoff geht die Replica an einen Ersatz-Node mit einem Hint (Vermerk), an wen sie eigentlich gehört. Nach Recovery wird nachgeliefert.

Wie kann Caching-Bias in Empfehlungssystemen reduziert werden?

Wie kann Caching-Bias in Empfehlungssystemen reduziert werden?

A

Nur populäre Inhalte cachen

Wie kann Caching-Bias in Empfehlungssystemen reduziert werden?

A

Nur populäre Inhalte cachen

B

Größeren Redis-Server nutzen

Wie kann Caching-Bias in Empfehlungssystemen reduziert werden?

A

Nur populäre Inhalte cachen

B

Größeren Redis-Server nutzen

C

Cache komplett deaktivieren

Wie kann Caching-Bias in Empfehlungssystemen reduziert werden?

A

Nur populäre Inhalte cachen

B

Größeren Redis-Server nutzen

C

Cache komplett deaktivieren

D

Epsilon-Greedy Exploration einbauen

Wie kann Caching-Bias in Empfehlungssystemen reduziert werden?

A

Nur populäre Inhalte cachen

B

Größeren Redis-Server nutzen

C

Cache komplett deaktivieren

D

Epsilon-Greedy Exploration einbauen

Erklärung: Ähnlich wie bei Reinforcement Learning kann zufällige Exploration sicherstellen, dass auch weniger populäre Inhalte eine Chance bekommen.

11

Diskussion

10 Minuten

Referenzen & Quellen

[1] DeCandia, G. et al. (2007). Dynamo: Amazon's Highly Available Key-value Store. SOSP '07.

[2] Everpure (2024). What Are Key-value Databases? everpuredata.com/knowledge/key-value-databases

[3] Redis Documentation. Commands Reference. redis.io/commands

[4] Brewer, E. (2000). Towards Robust Distributed Systems (CAP Theorem). PODC Keynote.

[5] Karger, D. et al. (1997). Consistent Hashing and Random Trees. STOC '97.

[6] Lamport, L. (1978). Time, Clocks, and the Ordering of Events. ACM Communications 21(7).

[7] European Parliament (2016). DSGVO/GDPR. Art. 17 -- Right to Erasure.

[8] Redis Ltd. Redis Cluster Specification. redis.io/docs/management/scaling

[9] redis-py Documentation. redis.readthedocs.io